02. Understanding HTTP & How Requests Work

1. Setting the Stage: The Waiter's Order Slip

In the previous note, we learned that the **API is like a Waiter** carrying messages between the Customer (Frontend) and the Kitchen (Backend Server).

Now, consider a practical question: **How does the customer actually tell the waiter what they want?**

If a customer just waves their arms, mumbles words, or speaks in an unknown language, the waiter cannot help them. The kitchen will either cook the wrong meal or reject the order completely.

To prevent confusion, every restaurant uses a **standardized Order Slip**:



Why does this printed notepad exist?

- The customer knows exactly what information they need to provide.
- The waiter knows how to carry it without losing details.
- The chef in the kitchen can read it instantly and start cooking.

In software, this standardized order slip is called an HTTP Request.

---

2. Why Do Computers Need a "Protocol"?

Humans have common sense. If you ask a friend: *"Coffee?"*, they understand you are offering them a drink based on context and tone.

Computers have **no common sense**.

- Your phone might run JavaScript on an iPhone built by Apple.
- The backend server might run Python on a Linux machine in a data center thousands of miles away.

Between these two devices, there is only a network cable. If they send raw, unstructured data, neither machine will know what the other is saying.

To talk successfully, both machines must agree on a strict rulebook:

1. Who speaks first?
2. How is the message structured?
3. How does the receiver understand the message?

That agreed-upon rulebook is called a Protocol.

What is HTTP?

HTTP (HyperText Transfer Protocol) is the universal protocol of the internet. It was built on one fundamental principle:

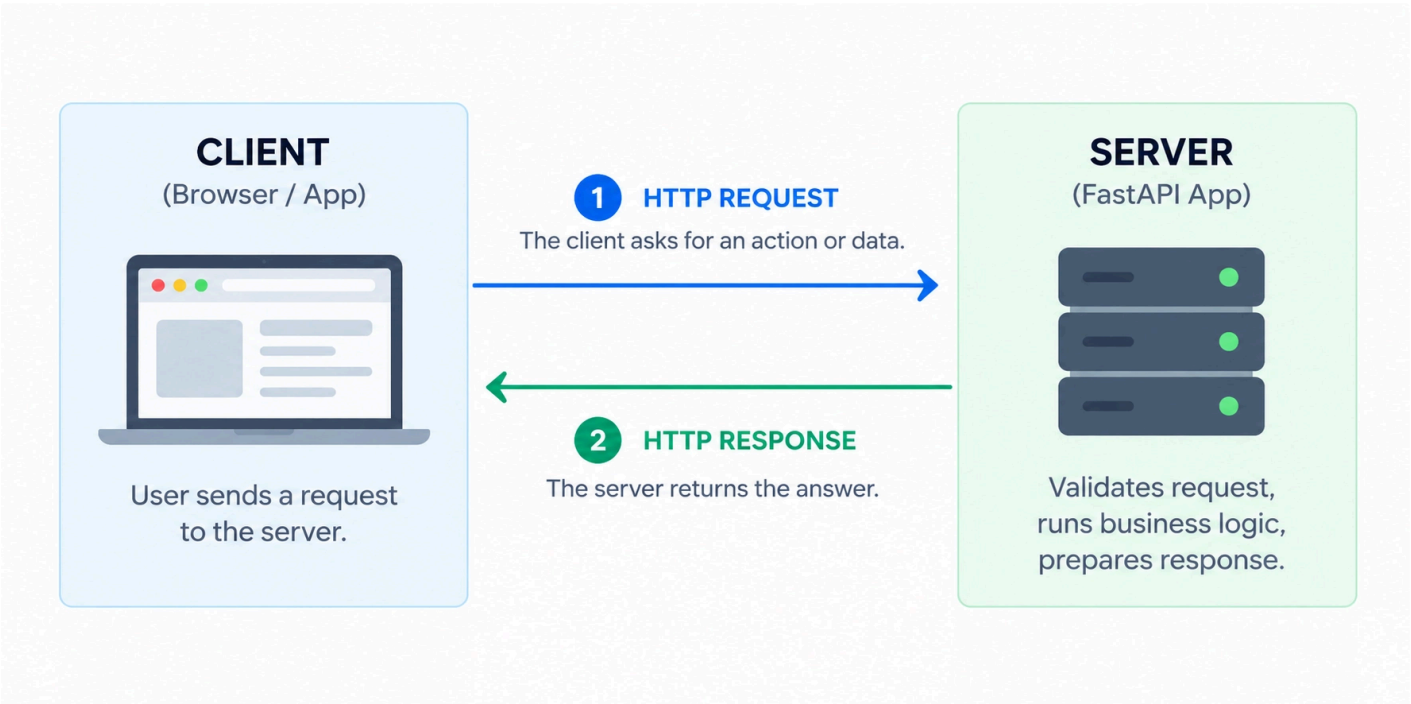
> **The Client asks a question (Request), and the Server answers (Response).**

> The server never speaks on its own; it only listens and responds when a client sends a request.

---

3. The Two-Step Cycle: Request and Response

Every interaction on the web is a strict 2-step conversation between the Client and the Server:

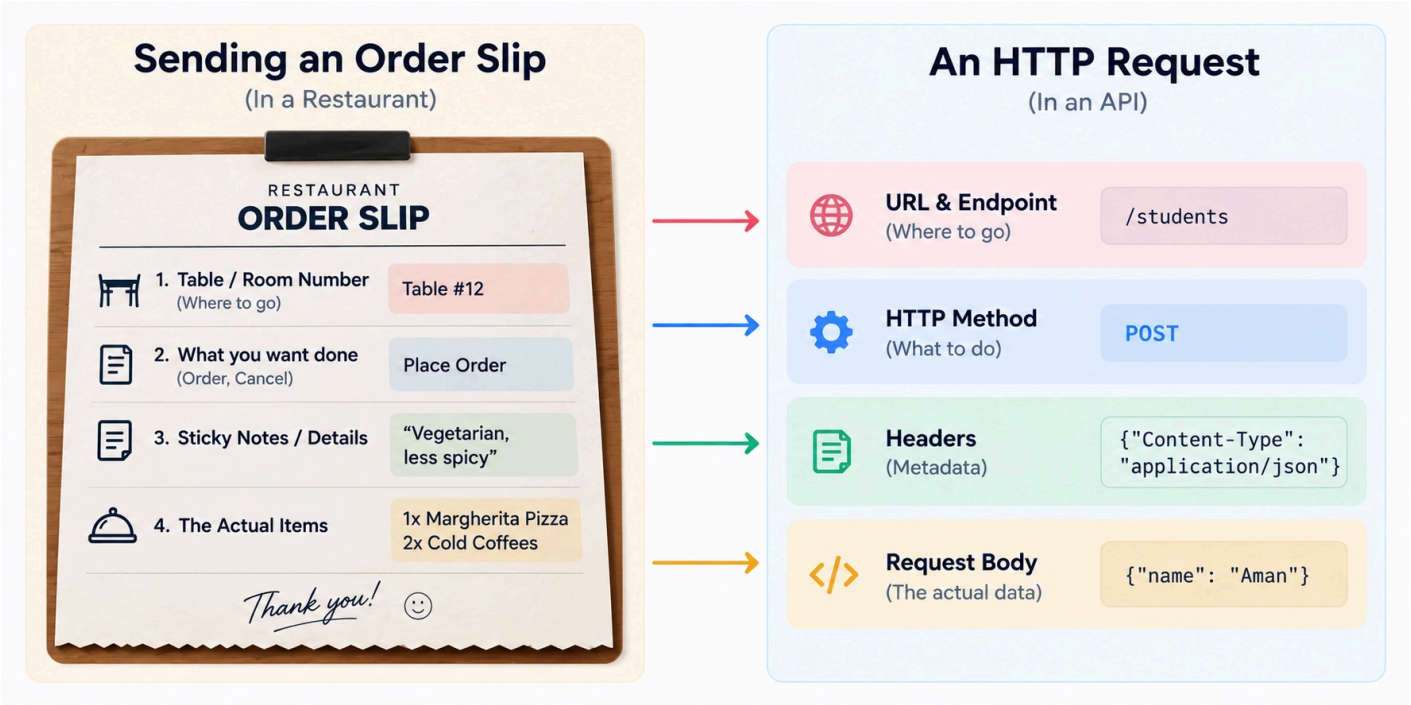


A backend server is passive: it sits idle waiting for incoming requests. The moment an HTTP request arrives, it processes the request, sends back an HTTP response, and goes back to waiting.

---

4. What is Inside an HTTP Request? (The 4 Parts)

When your frontend application asks the server for something, it bundles four simple pieces of information together into an **HTTP Request**:



1. URL & Endpoint (The Address)

- A server hosts many different resources (e.g., students, courses, payments).
- An **endpoint** is the specific door or room on the server you are visiting (e.g., `http://localhost:8000/students`).
- It tells the server *which exact resource* the request is meant for.

2. HTTP Method (The Action)

- Arriving at an address is not enough; the server needs to know what you want to do there.
- Are you asking to *view* information, *create* new information, or *delete* something? The method states your intention clearly.

3. Headers (The Metadata)

- Headers are like sticky notes or postal stamps on the outside of an envelope.
- They provide small extra details before the server reads the main content (such as the format of the data being sent).

4. Request Body (The Payload)

- When you are simply asking to view data, you don't need to send any form inside the envelope.
- But when you register as a new student or update your profile, you must submit details (name, age, course).
- This submitted information travels inside the **Request Body**.